Dynamic Hyperparameter Optimization During Training via PBT and Hypergradient Descent

Jordan Argel
Jordan Argel (jordanmatthew.argel@berkeley.edu)

Etaash Patel
Etaash Patel (etaashpatel@berkeley.edu)

Mark Pustilnik
Mark Pustilnik (pkmark@berkeley.edu)

Satyam Sharma
Satyam Sharma (satyam_sharma@berkeley.edu)

Abstract

We investigate whether small, compute-efficient within-run adjustments to hyperparameters can improve training of vision transformers under strict GPU-hour budgets. Concretely, we compare traditional grid-search with (i) a three-stage stage-wise grid, (ii) a cheap Population-Based Training–inspired perturb-and-select rule, and (iii) finite-difference hypergradient descent on CIFAR-10 using Tiny-ViT variants at two depths. Preliminary results suggest that low-cost within-run adaptation can improve performance beyond static-transferred hyperparameters while remaining computationally inexpensive.

1 Introduction

Selecting effective hyperparameters is central to training modern deep networks, but exhaustive tuning is increasingly infeasible as models and datasets scale. A popular strategy is *hyperparameter transfer*, where near-optimal hyperparameters are learned on a proxy model N' and then extrapolated to a larger target model N using theory or empirical scaling laws (Wen et al. [2025], Yang et al. [2023], Li et al. [2025a]). While these methods reduce search cost, they largely treat hyperparameters as fixed or as following pre-specified schedules across training.

Recent work challenges this static view. Some recent studies demonstrate that optimal values for some hyperparameters (learning rate, weight decay, and momentum) can evolve across training, and that dynamic schedules can potentially improve stability, memory footprint, or final accuracy (Marek et al. [2025], Kosson et al. [2025]). This motivates a simple question: *can we improve training by allowing small, compute-efficient adaptations of hyperparameters during training?*

In this project, we study three lightweight online HPO strategies: (1) a three-state grid search, (2) a cheap Population-Based Training (PBT)–inspired perturb-and-select rule, and (3) hypergradient descent, where hyperparameters are updated using numerical estimates of $\nabla_\theta \mathcal{L}_{\text{val}}$. All approaches operate under strict budget constraints and are evaluated on the CIFAR-10 dataset. Our goal is to characterize when and how online adaptation improves validation accuracy within given compute budgets, particularly as network-size varies.

Hypothesis: We hypothesize that small, compute-efficient online hyperparameter adjustments will yield higher validation accuracy than static transferred hyperparameters under equal compute, and that these gains will be more pronounced for deeper transformer models.

2 Background and Related Work

2.1 Problem setup and classical HPO

Hyperparameter Optimization (HPO) concerns selecting hyperparameters $\alpha \in A$ that minimize a model's validation loss. We formalize a model f_θ^α parametrized by both model parameters $\theta \in \mathbb{R}^p$ and hyperparameters $\alpha \in A$. A *training method* is a procedure that, given α, attempts to minimize training loss:

$$f_{\theta^*}^\alpha \approx \arg\min_{\theta \in \mathbb{R}^p} \mathcal{L}_{train}(f_\theta^\alpha). \tag{1}$$

HPO then aims to choose a hyperparameter bundle α that minimizes validation loss:

$$\text{HPO}(f_\theta^\alpha) \approx \arg\min_{\alpha \in A} \mathcal{L}_{validation}(f_{\theta^*}^\alpha). \tag{2}$$

Classical HPO treats the training procedure as a black box and searches over α using grid search, random search, Bayesian optimization, or multi-fidelity bandit strategies such as Hyperband (Franceschi et al. [2024], Li et al. [2018]). These methods typically require many independent training runs (full or partially completed), as evaluating a single candidate for α often requires a full or nearly full training run. As a result, such techniques often become computationally expensive as models and datasets scale (Wen et al. [2025], Franceschi et al. [2024]). This computational burden motivates two broad directions: transfer-based approaches that reuse information across model scales or proxies, and online (within-run) HPO methods that adapt hyperparameters during a single training trajectory. The next subsections summarize these directions and heuristics commonly used in practice.

2.2 Static HPO Methods

Many heuristic schedules remain popular because of their simplicity and low overhead. These include step decay, cosine annealing, cyclical learning rates and one-cycle policies (Smith and Topin [2017]), curriculum or dynamic dropout schedules (Morerio et al. [2017], Yan and

Shao [2024a]), and adaptive batch-sizing strategies (Lau and Others [2024]). Their primary advantage is that they can allow hyperparameters to evolve to a limited extent, while introducing essentially no auxiliary computation. However, fixed or hand-designed schedules cannot respond to task-specific or time-varying changes in the optimal hyperparameters, as the schedules and/or static values are predetermined. As a result, these heuristics may miss performance improvements that arise from adjusting learning rate, batch size, or dropout in response to evolving training dynamics.

Complementing heuristics, transfer-based HPO methods aim to amortize search cost by finding near-optimal hyperparameters $\widehat{\alpha}$ on a smaller proxy model $\hat{f}_{\omega}^{\alpha}$ (with $\omega \in \mathbb{R}^d$ and $d \ll p$). After hyperparameters $\widehat{\alpha}$ are found, they are mapped to the original model via a transfer function $t(\cdot)$ (Yang et al. [2023], Li et al. [2025a], Everett et al. [2024]). Constructions for $t(\cdot)$ include theoretical prescriptions, empirical scaling laws (e.g., power-law fits for hyperparameters like learning rate and batch size), and alignment metrics that predict transfer success (Yang et al. [2023], Li et al. [2025a], Everett et al. [2024]). Overall, transfer methods can substantially reduce search cost, and their reliability and performance can be further improved by incorporating measurements of alignment into transfer functions, as shown in (Everett et al. [2024]). However, transfer-based techniques can suffer from hyperparameter shifts, where the optimal set of hyperparameters either changes during training or depends strongly on depth or hidden dimensions. In these cases, a static transferred value may only be approximately correct early in training.

In both cases, we see HPO techniques that have been effective in training large models (Wen et al. [2025]), but might suffer from performance because they may not allow hyperparameters to adapt to task-specific or time-varying dynamics in training. Recent work supports this idea. Marek et al. [2025] suggests that adaptive choices of optimizer hyperparameters can improve stability in some regimes. Additionally, Kosson et al. [2025] argues that μP-related scaling rules inform only early-phase rates while time-varying adjustments later in training affect final performance more significantly. These observations motivate hybrid strategies that dynamically evolve hyperparameters over training. A potential application of such methods could be to start from transfer-derived values and then utilize small, inexpensive adjustments to hyperparameters during training. We call such techniques *online HPO techniques*.

2.3 Online HPO methods: hypergradients, population methods, and heuristics

A substantial literature develops algorithms for online HPO. Franceschi et al. [2024] survey several such techniques as part of a comprehensive literature review. We focus on two widely-used families of techniques: Population-Based methods (PBT and variants) and hypergradient Descent (HGD). Traditional online methods introduce adaptivity unavailable to static or transfer approaches; they adjust hyperparameters in response to training progress. However, this adaptivity often has a significant computational cost. Thus, the central tradeoff is adaptivity versus overhead, and lightweight variants are desirable when compute is limited, or when training is expensive.

In this paper we mainly study two families of online HPO methods: Population-Based Training (PBT) and hypergradient Descent (HGD). PBT, first introduced in Jaderberg et al. [2017a], maintains multiple workers that periodically exchange hyperparameters and model weights based on relative validation performance. Variants such as Population-Based Bandits (PB2) refine selection/mutation via bandit criteria for better sample efficiency (Parker-Holder et al. [2020]). PBT-style methods have been studied across reinforcement-learning and supervised settings, with differing population sizes and exploit/mutate schedules (Jaderberg et al. [2017a], Parker-Holder et al. [2020]). Note however that neither paper measures the performance of PBT in relation to model size or with respect to vision transformer networks.

The second family of techniques, HGD based methods, compute or approximate the gradient of validation loss with respect to hyperparameters and then use this gradient to update hyperparameters during training. The compact pseudocode below shows the pattern: a

standard model update followed by a (possibly approximate) hypergradient computation and a hyperparameter step.

Algorithm 1 hypergradient Descent (HGD)

Require: Initial model θ_0, hyperparams α_0, total steps T, model LR η, hyper LR γ
1: **for** $t = 1$ to T **do**
2: **Model step:** $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta \mathcal{L}_{\text{train}}(f_{\theta_{t-1}}^{\alpha_{t-1}})$
3: **hypergradient:** compute/approximate $g_t \approx \nabla_\alpha \mathcal{L}_{\text{val}}(f_{\theta_t}^{\alpha_{t-1}})$
4: **Hyper update:** $\alpha_t \leftarrow \alpha_{t-1} - \gamma\, g_t$
5: **end for**
6: **return** final (θ_T, α_T)

Baydin et al. [2017] introduced online hypergradient descent for learning-rate adaptation and demonstrated improvements on logistic regression and MLPs (Baydin et al. [2017]). While computing an exact hypergradient can be computationally expensive, subsequent works like (Donini et al. [2020]) try to address these issues by developing reasonable approximations and methods to scale HGD into higher dimensions. Despite the progress made in HGD, most of the papers we found evaluate on MLPs, CNNs, or RNNs rather than transformer pretraining, leaving open questions about performance computational cost for transformers. Additionally, none of the papers explore the relationship between model size and relative performance.

2.4 How this project is positioned

As seen in the previous subsections, previous research examining online HPO techniques often test on CNN, ResNet, or RNN contexts rather than transformers. While recent research has started to evaluate online HPO methods on transformers, most recent work does not evaluate the efficiency of these methods with respect to model size (Urteaga et al. [2023], Li et al. [2025b], Yan and Shao [2024b]).

Building on prior work, our project focuses on the intersection between model size, transformers, and online HPO techniques. More specifically, we focus on transformer architectures for image processing and evaluate three computationally lightweight online HPO strategies under strict compute budgets: (1) a three-state grid-search over a small discrete set of hyperparameter choices, (2) a cheap PBT–inspired perturb-and-select rule, and (3) HGD using inexpensive numerical estimates of $\nabla_\theta \mathcal{L}_{\text{val}}$. We use CIFAR-10 as a controlled testbed (Krizhevsky et al. [2009]), and evaluate multiple hyperparameter types.

Choice of Optimizer and hyperparameters: In all experiments we use the AdamW optimizer, following standard practice for Vision Transformers. We focus our adaptation on learning rate, weight decay, batch size, and dropout because prior work identifies these hyperparameters as having the strongest influence on transformer optimization dynamics, training stability, and generalization performance. These choices allow us to isolate the effects of lightweight online HPO on the most impactful components of the training process.

3 Problem Statement and Key Questions

We investigate whether allowing small online adjustments to hyperparameters improves performance under fixed compute and wall-clock budgets, and how these potential improvements might change with respect to model depth. Concretely, we address the following questions:

1. Does online adaptation initialized via transfer outperform fixed transferred hyperparameters at equal wall-clock time?
2. Which hyperparameters yield the highest marginal gain when adapted: learning rate, weight decay, batch-size, or droupout rate?
3. How do improvements vary with model scale when moving from proxy networks N' to deeper N?

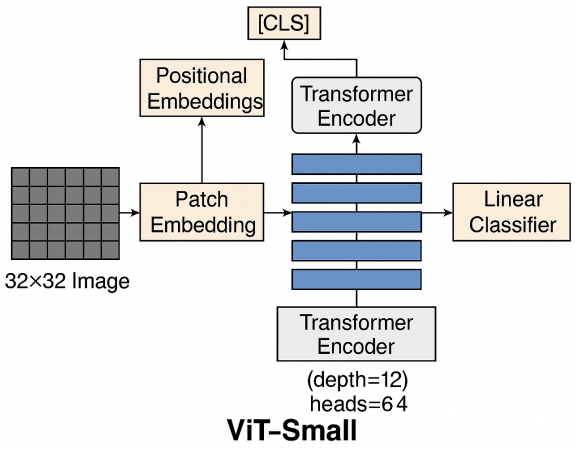


Figure 1: Schematic overview of the ViT-Small architecture used in our experiments.

4 Hypotheses

1. Lightweight online adaptation will match or exceed fixed-transfer baselines without exceeding our compute envelope.

2. The largest benefits will arise from adapting weight decay in early-to-mid training, followed by learning rate adjustments later.

3. Gains will increase with model depth, as larger models exhibit stronger temporal drift in optimal hyperparameter values.

5 Methods

5.1 Data and Architecture

All experiments use small vision transformers known as Vision Transformer Small (ViT-Small). It is a well-known model in the literature and is commonly used for benchmark comparison (Dosovitskiy [2020]). ViT-Small is configured with an embedding dimension of 384, 12 transformer encoder layers, 6 attention heads per layer, and an MLP expansion ratio of 4. Images are divided into non-overlapping 4×4 patches, yielding 64 tokens for 32×32 inputs, and a learnable class token is prepended, resulting in a sequence length of 65. Each encoder block follows a pre-norm design and contains multi-head self-attention and a two-layer feed-forward MLP, both with residual connections. Learnable positional embeddings are added to all tokens. The final representation is taken from the class token and passed through a linear classifier.

We evaluate all HPO methods on the image CIFAR-10 dataset (Krizhevsky et al. [2009]). This dataset includes 60,000 images, each with 32×32 color pixels. The images are divided into 10 classes and the goal is to predict the class of the image. We also randomly divide our dataset into 50,000 images for training and 10,000 images for validation.

All training runs are done over 90 epochs. For additional implementation details, please refer to the appendix.

5.2 Baseline

We first perform a basic grid search using our model. In order to obtain results in limited access to compute we search over the grid defined by the following Cartesian product given

by

$$\begin{aligned} \text{lr} &\in \{1 \times 10^{-4},\, 3.3 \times 10^{-4},\, 1 \times 10^{-3}\}, \\ \text{wd} &\in \{0.02,\, 0.07,\, 0.15\}, \\ \text{bs} &\in \{128,\, 256,\, 512\}, \\ \text{dpr} &\in \{0.0,\, 0.1,\, 0.2\}. \end{aligned}$$

5.3 Proof of Concept: 3-Stage Grid Search

We first consider a method that is conceptually simple, easy to implement, and incurs little to no extra compute. We divide training into 3 stages: (1) epochs 0-15, (2) epochs 20-45, (3) epochs 50-90. Epochs in between the three stages are used as warmup epochs so the optimizer can adapt to the new landscape caused by shifting to different hyperparameters. We then perform independent grid searches during each stage.

5.4 Online Methods [runs not finished]

For our first online method, we implement a simplified version of Population-Based Training (PBT) (Jaderberg et al. [2017b]). Multiple models are trained in parallel, each with its own perturbed values around the nominal of some selected hyperparameter (e.g., dropout rate, batch size). These run in parallel for a few epochs, and at the end of these epochs the model with the highest validation accuracy is selected. The nominal value of the hyperparameter is then updated, and the process repeats (with different hyperparameters) until the end of training. An overview of this method is given below.

Algorithm 2 Perturb-and-Select Hyperparameter Update (Every n Iterations)

Initialize: model M, hyperparameter vector α, number of steps n
for iteration $t = 1, 2, \ldots$ **do**
  if $t \bmod n = 0$ **then** ▷ Perform update every n iterations
    Select a hyperparameter α_j to update
    Generate k perturbed values $\{\alpha_j^{(1)}, \ldots, \alpha_j^{(k)}\}$ ▷ e.g., multiplicative $\pm 25\%$
    for $i = 1$ to k **(in parallel) do**
      Create a copy M_i with $\alpha_j = \alpha_j^{(i)}$
      Train M_i for n iterations
      Evaluate M_i on the validation set to obtain validation loss l_i
    end for
    $i^* \leftarrow \arg\min_i l_i$
    Update nominal hyperparameter: $\alpha_j \leftarrow \alpha_j^{(i^*)}$
    Update model $M \leftarrow M_{i^*}$
    Optionally reset or resynchronize LR scheduler
  end if
end for

This method is simple and easy to integrate within training loops. However, it can be computationally expensive as k increases.

For our second HPO method, we implement a lightweight hyperparameter adaptation approach inspired by classical hypergradient descent (Baydin et al. [2017]) and MARTH-style online HPO (Donini et al. [2020]). The main idea is to gradually evolve hyperparameters without significantly increasing computational cost. To achieve this, we periodically compute numerical gradients for continuous hyperparameters such as learning rate, weight decay, and dropout rate. Every k iterations, we perturb each hyperparameter of interest, clone the model, and train it for a few (here, 5) batches under each perturbed value. We chose 5 batches to balance sufficient local sampling for accurately approximating the hypergradient with keeping gradient noise low. The resulting validation losses are then used to

compute the numerical gradient, which we use to update the hyperparameters with a fixed step size before resuming standard training. Pseudocode for this method is provided below.

Algorithm 3 Perturb-and-Probe Hypergradient Descent

Require: Model parameters w; hyperparameters $\alpha = (\alpha_1, \ldots, \alpha_h)$; outer-loop interval K; perturbation size δ; hyper-LR η_h; number of probe batches B

for epoch $t = 1, 2, \ldots$ **do**
  update w for all minibatches in epoch t ▷ Train normally
  if $t \bmod K = 0$ **then** ▷ Run hyperparameter update every K epochs
    for each hyperparameter α_i **do**
      $\alpha_i^{(+)} \leftarrow \alpha_i + \delta$, $\alpha_i^{(-)} \leftarrow \alpha_i - \delta$ ▷ Define positive and negative perturbations
      $w^{(+)} \leftarrow w$, $w^{(-)} \leftarrow w$ ▷ Clone model parameters for both perturbations
      for $b = 1$ to B **do** ▷ Run few batches to approximate hypergradient
        update $w^{(+)}$ using $\alpha_i^{(+)}$ ▷ Step positive clone
        update $w^{(-)}$ using $\alpha_i^{(-)}$ ▷ Step negative clone
      end for
      $L_i^{(+)} \leftarrow \text{ValLoss}(w^{(+)}, \alpha_i^{(+)})$, $L_i^{(-)} \leftarrow \text{ValLoss}(w^{(-)}, \alpha_i^{(-)})$ ▷ Compute validation losses
      $d_i \leftarrow \frac{L_i^{(+)} - L_i^{(-)}}{2\delta}$ ▷ Compute finite-difference hypergradient
      $\alpha_i \leftarrow \alpha_i - \eta_h \, d_i$ ▷ Update hyperparameter
    end for
  end if
end for

This method adds less computational overhead (1–2 extra forward passes per epoch) and complements the exploratory PBT-style method.

5.5 Ablations [runs not finished]

To better analyze the relationships between online HPO methods and model size, we repeat sections 5.2-5.5 on a larger model N', where everything is held the same as in our original model, but network depth is increased from 12 to 30.

5.6 Compute Budget

All experiments run on personal GPUs and limited departmental compute. We track wall-clock time, GPU-hours, and overhead introduced by each adaptive method.

6 Results

6.1 Baseline Performance [Preliminary]

Table 1: Grid-Search Hyperparameter Tuning on CIFAR — Top 10

Rank	Accuracy	LR	WD	BS	DPR
1	85.33%	0.00033	0.07	128	0.2
2	84.97%	0.00033	0.15	128	0.2
3	84.97%	0.00033	0.15	128	0.1
4	84.95%	0.00033	0.07	256	0.2
5	84.88%	0.00033	0.02	128	0.2

Table 2: Leaderboard results across phases

(a) Phase 1 (baseline) leaderboard (best test accuracy)

Rank	Accuracy	LR	WD	BS	DPR
1	76.58%	0.00033	0.07	128	0.2

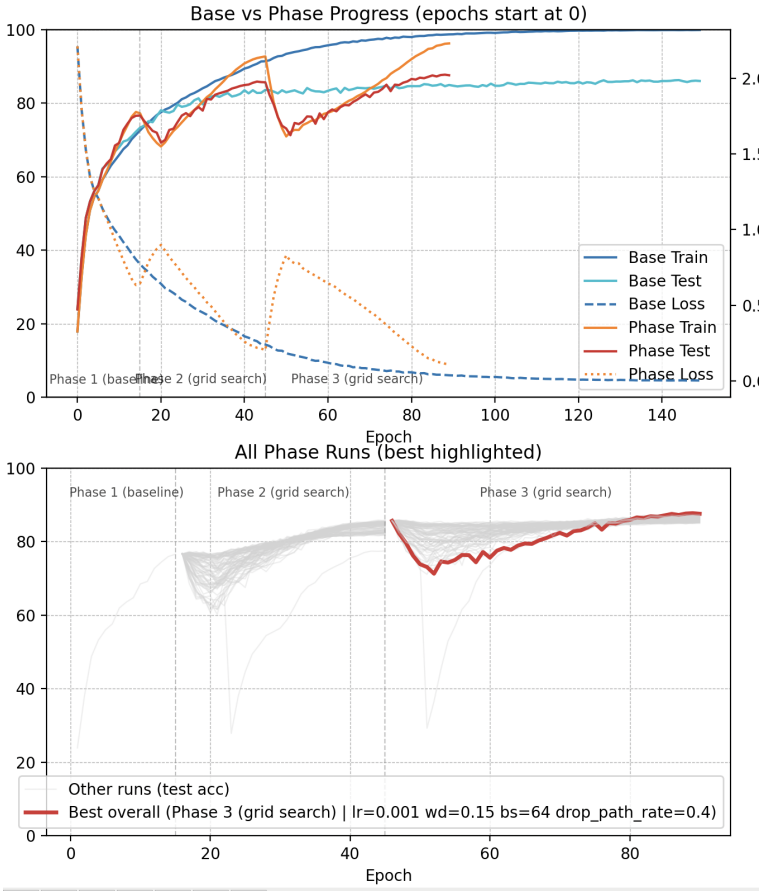
(b) Phase 2 (grid search) leaderboard (best test accuracy)

Rank	Accuracy	LR	WD	BS	DPR
1	85.87%	0.00033	0.07	64	0.4
2	85.64%	0.00033	0.02	64	0.4
3	85.56%	0.001	0.15	128	0.4
4	85.53%	0.001	0.07	128	0.4
5	85.52%	0.001	0.15	128	0.1

(c) Phase 3 leaderboard (best test accuracy)

Rank	Accuracy	LR	WD	BS	DPR
1	87.75%	0.001	0.15	64	0.4
2	87.70%	0.001	0.07	64	0.4
3	87.50%	0.001	0.07	128	0.4
4	87.47%	0.001	0.07	64	0.2
5	87.41%	0.001	0.07	256	0.4

6.2 3-Stage Grid search Results [Preliminary]



6.3 Online Adaptation Performance [running not finished]

Insert plots for:

- Accuracy over epochs
- Calibration metrics (ECE)
- Hyperparameter trajectories

6.4 Ablations [not finished]

7 Discussion [to be done after results come in]

7.1 Effectiveness of Online Adaptation

Summarize whether PBT and hypergradient descent improved over fixed transfer and under what conditions.

7.2 Which hyperparameters Matter Most?

7.3 Tradeoff Between Accuracy and Compute

7.4 Relationship with Model Size

7.5 Limitations and Extensions

References

Kaiyue Wen, David Hall, Tengyu Ma, and Percy Liang. Fantastic pretraining optimizers and where to find them. 2025. URL https://arxiv.org/abs/2509.02046.

Greg Yang, James B. Simon, and Jeremy Bernstein. A spectral condition for feature learning. 2023. URL https://arxiv.org/abs/2310.17813.

Houyi Li, Wenzhen Zheng, Qiufeng Wang, Hanshan Zhang, Zili Wang, Shijie Xuyang, Yuantao Fan, Zhenyu Ding, Haoying Wang, Ning Ding, Shuigeng Zhou, Xiangyu Zhang, and Daxin Jiang. Predictable scale: Part i, step law – optimal hyperparameter scaling law in large language model pretraining, 2025a. URL https://arxiv.org/abs/2503.04715.

Martin Marek, Sanae Lotfi, Aditya Somasundaram, Andrew Gordon Wilson, and Micah Goldblum. Small batch size training for language models: When vanilla sgd works, and why gradient accumulation is wasteful, 2025. URL https://arxiv.org/abs/2507.07101.

Atli Kosson, Jeremy Welborn, Yang Liu, Martin Jaggi, and Xi Chen. Weight decay may matter more than μp for learning rate transfer in practice. 2025. URL https://arxiv.org/abs/2510.19093.

Luca Franceschi, Michele Donini, Valerio Perrone, Aaron Klein, Cédric Archambeau, Matthias Sänger, Massimiliano Pontil, and Paolo Frasconi. Hyperparameter optimization in machine learning. 2024. URL https://arxiv.org/abs/2410.22854.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. In *Journal of Machine Learning Research (JMLR)*, 2018.

Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Data*, 2017.

Pietro Morerio, Andrea Cavallaro, Francesco Tortorella, and Andrea Fusiello. Curriculum dropout. In *Computer Vision and Pattern Recognition (CVPR)*, 2017.

Author Yan and Author Shao. Dynamic dropout for transformers. In *arXiv preprint arXiv:2401.XXXXX*, 2024a.

Author Lau and Others. Principled adaptive batch size schedules for language model training. In *arXiv preprint arXiv:2402.XXXXX*, 2024.

Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, and Jeffrey Pennington. Scaling exponents across parameterizations and optimizers, 2024. URL https://arxiv.org/abs/2407.05872.

Max Jaderberg, Wojciech M Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Population based training of neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017a.

James Parker-Holder, Zoubin Ghahramani, and Jonathan Frankle. Population based bandits: A new bandit approach for population based training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Atilim Gunes Baydin, Ryan Cornish, David Martinez-Rubio, Mark Schmidt, and Frank Wood. Online learning rate adaptation with hypergradient descent. In *International Conference on Learning Representations (ICLR)*, 2017.

Michele Donini, Luca Franceschi, and Massimiliano Pontil. Marthe: A meta-learning approach to hyperparameter optimization. In *IJCAI 2020*, 2020.

Iñigo Urteaga, Moulay-Zaïdane Draïdia, Tomer Lancewicki, and Shahram Khadivi. Multi-armed bandits for resource efficient, online optimization of language model pre-training: the use case of dynamic masking. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10609–10627. Association for Computational Linguistics, 2023. URL https://aclanthology.org/2023.findings-acl.675.

Zheng Li, Jerry Cheng, and Huanying Helen Gu. Sequential policy gradient for adaptive hyperparameter optimization. *arXiv*, abs/2506.15051, 2025b. URL `https://arxiv.org/abs/2506.15051`.

Hanrui Yan and Dan Shao. Enhancing transformer training efficiency with dynamic dropout. *arXiv*, abs/2411.03236, 2024b. URL `https://arxiv.org/abs/2411.03236`.

Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

Max Jaderberg, V. Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z. Leibo, David Silver, and Koray Kavukcuoglu. Population based training of neural networks. In *Advances in Neural Information Processing Systems*, 2017b.

A Supplementary Material

A.1 Base Results

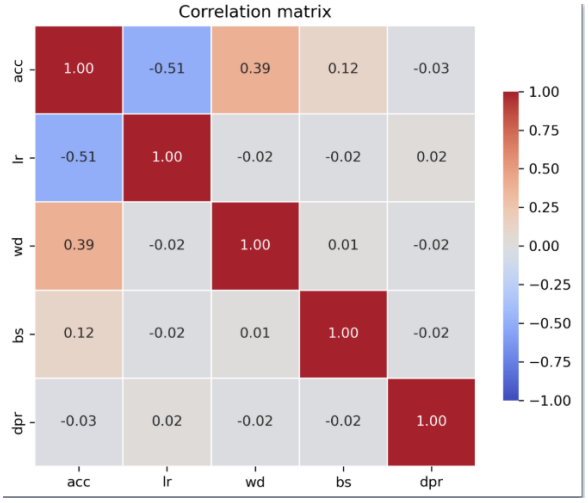


Figure 2: Correlation matrix of validation accuracy and hyperparameters for the baseline grid search.

A.2 Implementation Details

Table 3: Implementation and reproducibility details.

Item	Value / Notes
Repository	`https://github.com/EtaashP/CS182-Final-Project.git`
Hardware	NVIDIA RTX PRO 6000 Blackwell (97 GB VRAM); 186 GiB system RAM
Wall-clock budget	48 GPU-hours per experiment (hard cap)
OS / Drivers	Ubuntu 22.04.5 LTS; NVIDIA driver 580.95.05; CUDA runtime 13.0 (PyTorch CUDA build 12.8)
Python	Python 3.11
Frameworks	PyTorch 2.9.1+cu128; torchvision 0.24.1+cu128; numpy 2.2.6
Random seeds	{42}; deterministic flags disabled (we allow CUDA nondeterminism).
Datasets	CIFAR-10 (Krizhevsky et al. [2009]); standard 90% train, 10% validation split.
Data augmentation	RandomCrop(32,4), RandomHorizontalFlip, Cutout(16).
Model architectures	TinyViT and variants; see Section 5.
hyperparameters	LR, weight decay, batch size, dropout, drop-path rate
Warmup	5-epoch linear warmup applied before cosine decay.
Training steps	90 epochs; early stopping disabled.
Scheduler	All methods use the same cosine LR schedule. If a method changes LR abruptly (e.g., PBT exploit), the scheduler is reset. For drop rates, stochastic depth uses linear layerwise decay up to the chosen drop-path rate.
Precision	Mixed precision (AMP) enabled for all runs.
Optimizer (AdamW)	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$; decoupled weight decay
Repro command	`python run.py --config configs/tinyvit-depth6.yaml --seed 42`
License	N/A